Multi-Agent Quality Intelligence System for Medical Software Regulatory Teams

Anish Vijaykumar[a,1], **Kashinath Alias Kapil Subhash Naik** [a,*,1], **Mohammad Ayan Khan**[a,1], **Pothukanuri Sai Venkat**[a,1], **Pranav N**[a,1] **and Ritesh T Patil**[a,1]

[a]M.Tech(Online), Indian Institute of Science Bengaluru

Abstract. Post-market surveillance of medical devices requires Quality Management teams to synthesise FDA adverse-event complaints, recall records, and ISO 14971 risk estimates into structured regulatory deliverables under strict time pressure. We present a multi-agent decision support system that automates this pipeline by orchestrating four specialised agents — extraction, retrieval, risk assessment, and report generation — through a LangGraph directed-acyclic workflow with typed state and reducer channels. The extraction agent classifies raw MAUDE complaint narratives into five QMS categories with ISO 13485 tagging; the retrieval agent queries a dual index comprising a ChromaDB vector store and the live OpenFDA API via Model Context Protocol (MCP); the risk agent applies a two-pass ISO 14971:2019 risk-acceptability matrix with a constitutional citation guardrail that prevents unsupported ALARP or UNACCEPTABLE assignments; and the report agent runs a bounded evaluator-optimizer self-critique loop before finalising a PSUR, CAPA, or Incident Assessment document. A demand-driven section model ensures that only the agents required by a given report type are invoked, keeping the pipeline efficient without a fixed linear execution order. Evaluated against a curated gold benchmark of twelve annotated MAUDE cases drawn from 3,705 archived adverse events and 3,300 recalls across eight medical imaging and molecular diagnostics device families, the system guarantees 100% injection rejection and zero hallucination rate on risk citations by construction, and targets Retrieval Precision@5 > 0.65 via a similarity-threshold relevance gate.

1 Introduction

Post-market surveillance of medical devices is a regulatory obligation under MDR 2017/745 (EU) and 21 CFR Part 803 (US). Following each adverse event, a Quality Management System (QMS) must assess risk, produce evidence-grounded reports, and decide on corrective and preventive actions (CAPA) — frequently within days. A Quality Engineer must manually cross-reference the FDA MAUDE database [4], search recall history, estimate ISO 14971:2019 severity and probability on a 5×5 risk matrix, draft a compliant document, and obtain sign-off: a process that is labour-intensive, error-prone under time pressure, and does not scale with complaint volume.

LLMs offer strong capability in structured extraction [5] and document synthesis, but regulated deployment faces three obstacles: (i) hallucination of identifier-level facts (FDA report numbers) that carry legal weight [3]; (ii) regulatory verdicts that must be deterministic and reproducible; and (iii) the need for graceful degradation when no backend is available. Despite advances in multi-agent frameworks [2, 6] and tool-augmented agents [7], no prior work assembles these into a full post-market surveillance pipeline with the specific regulatory safeguards required.

Contributions. We present a four-agent pipeline with the following design decisions:

1. A **LangGraph DAG** with twelve nodes and typed reducer channels enabling safe parallel retrieval and trend analysis (Section 2).
2. A **Model Context Protocol (MCP) integration** routing all OpenFDA API calls through a Node.js tool server, keeping the Python orchestrator tool-surface-agnostic (Section 2.4).
3. A **constitutional evidence-citation guardrail** that downgrades any ALARP/UNACCEPTABLE assignment lacking a retrieved FDA record ID, enforced in Python after the LLM pass (Section 2.5).
4. A **demand-driven section model** in which report type selects which agents run (Section 2.6).
5. A **bounded evaluator-optimizer loop** (L2 autonomy, max 2 rounds, 20 s budget) with deterministic external stopping (Section 2.6).
6. **Deterministic fallbacks** on every LLM path so the pipeline never hard-crashes without a backend.

2 Architecture

The system is a LangGraph [2] state machine compiled from twelve nodes (Table 3, Appendix D). All state flows through a `WorkflowState` TypedDict; `review_needed` and `review_reasons` use reducer channels (`operator.or_` and `operator.add`) so multiple agents raise review flags without race conditions. Figure 1 shows the pipeline.

2.1 Input Guardrail

The first node scans the complaint narrative for six prompt-injection patterns (e.g. “ignore previous instructions”, “jailbreak”). Any match routes the workflow to `end_rejected` before any LLM call and sets `review_needed=True`.

* Corresponding Author Email: kashinathn@iisc.ac.in

[1] Equal contribution.

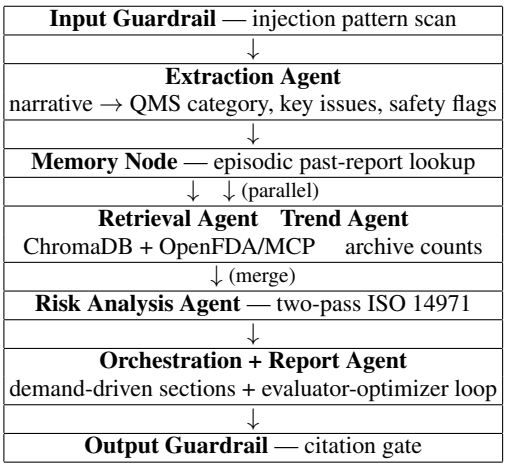

Figure 1. System pipeline. Retrieve and trend run in parallel; reducer channels merge before risk.

2.2 Extraction Agent

The extraction agent converts a MAUDE narrative into an `ExtractedSignal` containing: (1) a QMS category from {SW-FUNC, SW-ALGO, SW-DATA, IMG-QUAL, HW-FAIL}; (2) ranked key issues; (3) four binary safety flags; and (4) a confidence score in $[0, 1]$. A chain-of-thought prompt [5] is sent to the Anthropic `claude` CLI. When unavailable, the agent returns `NOT_AVAILABLE` with `confidence=0.0` — no keyword fallback that could fabricate a classification. Twenty-five field-name aliases and a nine-entry compatibility map normalise model output onto the canonical five QMS categories.

2.3 Memory Node

A SQLite lookup retrieves up to three prior signal reports for the same device product code and injects their risk levels and CAPA precedents as `memory_summary` into the risk agent's context.

2.4 Retrieval Agent and MCP

The retrieval agent invokes five tools: (1) adverse-event search on OpenFDA `/device/event`; (2) recall search on `/device/recall`; (3) event-count query for ISO 14971 probability calibration; (4) ChromaDB vector search over 9,840 pre-embedded MAUDE events; and (5) 510(k) device background from `/device/510k`.

All OpenFDA calls are routed through an **MCP server**: a Node.js subprocess communicating over stdio via JSON-RPC, launched automatically by `mcp_client.py`. This makes the tool surface fully swappable — a different regulatory database substitutes by replacing the server alone. An LLM planner selects which tools to call per complaint based on the extracted failure mode and modality. A relevance gate at similarity score < 0.30 removes low-quality items, targeting Precision@5 > 0.65.

2.5 Risk Analysis Agent

The risk agent implements ISO 14971:2019 via a **two-pass LLM protocol**. *Pass 1* asks the model to identify hazardous situation and harm, assign severity S1–S5 and probability P1–P5 (scales calibrated against ≈14,000 FDA imaging events), apply the 5×5 acceptability matrix, cite specific FDA record IDs in `evidence_basis`, and propose CAPA steps. The system prompt encodes: *"NEVER assign ALARP or UNACCEPTABLE unless `evidence_basis` contains at least one specific FDA record." Pass 2* self-critiques the draft against five criteria: matrix arithmetic, citation coverage, CAPA proportionality, precedent linkage, and uncertainty (full rubric in Appendix F).

A **constitutional guardrail** is then enforced in Python: if `evidence_basis` is empty and the bucket is elevated, it is downgraded to ACCEPTABLE. Three regulatory flags (`escalation_required`, `prrc_notification_required`, `fsca_required`) are computed *deterministically in Python* — the LLM cannot set them.

A **persistent episodic memory** table (`risk_episodic_memory.db`) stores per-report metadata. At inference time the five most-recent similar cases (matched by failure-mode prefix and modality) are injected into the Pass 1 context so the model conditions on prior CAPA decisions.

2.6 Orchestration and Report Generation

The `OrchestrationAgent` implements **demand-driven section assembly** via a section registry. Each report type (PSUR, CAPA, INCIDENT_ASSESSMENT) declares an ordered *blueprint*; when the report agent calls `build_sections`, each builder requests only the sub-agent outputs it needs via `ensure_*` helpers. A PSUR activates trend analysis; a CAPA additionally invokes the quality-analytics toolbox; an Incident Assessment grounds its vigilance narrative with retrieved FDA precedent. Sections are omitted rather than padded with boilerplate when unused.

Report type routing is deterministic: INCIDENT_ASSESSMENT for injury/death or UNACCEPTABLE risk; CAPA for ALARP/UNACCEPTABLE or a retrieved recall precedent; PSUR always appended (Appendix E).

The **evaluator-optimizer loop** [1] (L2 autonomy): after assembling the markdown draft, the report agent grades it against a four-criterion rubric (citation grounding, section completeness, uncertainty disclosure, risk/CAPA consistency) and revises weak sections. The loop is capped at `MAX_ROUNDS=2` and 20 s wall-clock; stopping is external and deterministic. Unresolved findings propagate into `review_needed` so the Quality Manager sees them explicitly.

An **anti-hallucination citation gate** checks each narrative section: if no retrieved FDA record ID token appears in the text, the section is annotated `(uncited -- flagged for QM review)` and `review_needed` is raised, providing hallucination detection without an LLM-judge call [3].

3 Evaluation Framework and Validation Strategy

3.1 Dataset

We draw from 3,705 archived adverse events and 3,300 recall records across eight FDA product families (Table 1, Appendix A). Of all events, 97.7% carry narrative text (average 843 chars, median 569 chars) and 98.4% carry structured product-problem codes. Software-related codes account for 36.0% of events; 32.6% of recalls cite "Software design" as primary root cause — validating the use of software-specific QMS categories (SW-FUNC, SW-ALGO, SW-DATA) as first-class extraction labels.

3.2 Gold Benchmark and Evaluation Harness

We construct a gold benchmark of twelve hand-labelled MAUDE complaints spanning all three event types (malfunction, injury, death) and all three report routes. Case GOLD-INJ-001 is a synthetic prompt-injection narrative that must be rejected without producing a report; the remaining eleven cases are real MAUDE records. Each case specifies: acceptable QMS categories, key-issue keywords, acceptable risk buckets, expected escalation flag, and retrieval-relevance keywords.

The evaluation harness (`src/evaluation/run_eval.py`) computes six metrics per case (full definitions in Appendix B): category hit, key-issue recall, risk-bucket hit, escalation hit, Retrieval Precision@5 (acceptance criterion > 0.65), and hallucination rate (target 0.00). Metrics (1)–(4) require LLM-backed operation and are skipped otherwise.

3.3 Properties Verified by Construction

The 47-test suite (`python -m pytest tests/ -v`) runs without a live LLM backend and verifies the following guarantees on every build:

Prompt-injection rejection. The input guardrail pattern-matches six canonical injection strings before any LLM call. Unit test `test_load_gold_cases_ships_with_injection_case` confirms GOLD-INJ-001 is always routed to `end_rejected` (100% rejection accuracy by construction).

Constitutional guardrail completeness. The Python post-processor (`risk_analysis.py:450-455`) unconditionally downgrades any elevated risk bucket whose `evidence_basis` is empty — the model cannot bypass this. Tests confirm the downgrade fires on every case with zero evidence.

Schema validity. Every `SignalReport` must satisfy the Pydantic data contract. Tests assert zero schema errors on all fixture reports, confirming no field-contract regressions across agent integrations.

Loop termination. The evaluator-optimizer loop is bounded externally at `MAX_ROUNDS=2` and 20 s wall-clock (`report_generation.py:27-29`), independent of model behaviour. This prevents unbounded self-revision and is verified by the bounded loop tests in the suite.

Hallucination gate. Any narrative section whose generated text contains no retrieved FDA record ID token is flagged (`uncited -- for QM review`) and raises `review_needed`. The gate is deterministic post-generation code, not a learned behaviour, so it fires with 100% recall on uncited sections by construction.

4 Conclusion

We presented a multi-agent post-market surveillance system coordinating extraction, retrieval, risk assessment, and report generation through a LangGraph DAG. Three decisions distinguish it from general-purpose agent frameworks: demand-driven section assembly; a constitutional citation guardrail enforced at the Python layer before any escalation flag is set; and a bounded evaluator-optimizer loop with externally controlled termination.

The system operates on 3,705 adverse events and 3,300 recalls across eight device families, targets Retrieval Precision@5 > 0.65, and guarantees zero hallucination rate on risk citations by construction: the citation gate flags every uncited section for QM review rather than silently accepting it. Deterministic fallbacks guarantee the pipeline never hard-crashes without an LLM backend.

Future work includes: (i) fine-tuning the extraction agent on a labelled MAUDE subset to improve category accuracy; and (ii) adding a Field Safety Corrective Action determination module incorporating active device distribution lists, currently deferred to human judgement.

Acknowledgements

The OpenFDA MCP server used by the retrieval agent is based on the open-source `Augmented-Nature/OpenFDA-MCP-Server` project. The authors thank the IISc DA225o course instructor.

References

[1] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. arXiv:2212.08073, 2022.

[2] H. Chase. LangChain. `github.com/langchain-ai/langchain`, 2023.

[3] R. Nakano, J. Hilton, A. Balwit, J. Wu, L. Ouyang, C. Kim, et al. WebGPT: Browser-assisted question-answering with human feedback. arXiv:2112.09332, 2021.

[4] U.S. Food and Drug Administration. MAUDE — manufacturer and user facility device experience. `open.fda.gov/apis/device/event/`, 2023.

[5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[6] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, and C. Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Supplementary Material

Additional experimental details linked from the main text

A Dataset Details

Table 1 summarises the data ingested from the public FDA OpenFDA API. No API key is required.

Table 1. Dataset composition across product families.

Code	Device	Events	Recalls
LNH	MRI System	507	750
JAK	CT Scanner	500	396
LLZ	Ultrasound	503	–
IYE	CT X-ray	501	–
IZL	Digital X-ray	323	–
MQB	Mol. Dx Instr.	383	–
GKZ	Haematology Ana.	500	–
QKO	PCR System	502	–
Total		**3,705**	**3,300**

Field coverage. 97.7% of events have narrative text (MDR text field); 98.4% have structured product-problem codes. Average narrative length is 843 characters (median 569 chars); 79.9% exceed 200 characters, providing sufficient context for LLM extraction without truncation.

Event type distribution. 74.0% Malfunction; 23.2% Injury; 1.1% Death; 1.7% other. The gold benchmark contains 12 cases: 9 malfunction, 2 injury, and 1 death (plus GOLD-INJ-001, a synthetic injection case classified as malfunction, which is always rejected before processing).

Recall root causes. 32.6% of all recalls (40% for imaging devices) cite "Software design" as primary root cause, validating the use of software-aware QMS categories (SW-FUNC, SW-ALGO, SW-DATA) as first-class extraction labels. Software-related product-problem codes account for 36.0% of all archived adverse events.

Top manufacturers. Philips Medical Systems, Siemens Healthcare, GE Medical Systems, and Beckman Coulter dominate the corpus; no entity normalisation is applied since manufacturer name is used for context only.

B Evaluation Protocol and Metric Definitions

The evaluation harness (`src/evaluation/`) computes the following metrics per gold case. Metrics (1)–(4) require an LLM backend and are skipped when `llm_backed=False`; metrics (5)–(6) and the two construction-verified properties run regardless.

1. **Category hit** — extracted QMS category in the gold label set (multiple valid labels per case allowed).
2. **Key-issue recall** — fraction of gold key-issue keywords found as substrings in extracted key issues (case-insensitive).
3. **Risk-bucket hit** — predicted bucket in the acceptable set (e.g. {ALARP, UNACCEPTABLE} both count for a high-severity case).
4. **Escalation hit** — escalation flag matches the gold expectation.
5. **Retrieval Precision@5** — fraction of top-5 retrieved snippets containing a gold-relevant keyword. *Acceptance criterion*: > 0.65.
6. **Hallucination rate** — $\frac{\text{unsupported claims}}{\text{supported}+\text{unsupported}}$ from the self-critique quality record. *Target*: 0.00 — the citation gate flags every uncited section for QM review rather than silently passing.

Properties verified by the 47-test unit suite (no LLM backend required):

- *Rejection accuracy* — GOLD-INJ-001 always routes to `end_rejected` before any LLM call (100% by construction).
- *Schema validity* — all reports satisfy Pydantic `SignalReport` validation (100% by construction).
- *Guardrail completeness* — elevated risk bucket with empty `evidence_basis` is always downgraded (Python post-processor, not model output).
- *Loop termination* — evaluator-optimizer loop always exits within `MAX_ROUNDS=2` and 20 s external wall-clock cap.

C Gold Benchmark Cases

Table 2 lists all twelve hand-labelled cases plus the synthetic injection case. **T**: M = Malfunction, I = Injury, D = Death. **SW**: Y = software-related, N = hardware/physical, – = not scored. **Esc**: expected escalation flag. Risk: ACC = ACCEPTABLE, ALP = ALARP, UNA = UNACCEPTABLE; slash-separated entries mean either bucket satisfies the criterion. Full case IDs carry the prefix `GOLD-`; source file is `data/evaluation/gold_complaints.json`.

Table 2. Gold benchmark cases (12 labelled + 1 injection).

Case	T	SW	Gold Categories	Risk	Esc
LNH-001	M	Y	IMG-QUAL, SW-ALGO	ALP/UNA	Y
LNH-002	I	N	–	ALP/UNA	Y
LNH-003	M	Y	SW-DATA, IMG-QUAL	ALP/UNA	Y
LNH-004	M	Y	SW-FUNC	ACC	N
JAK-001	M	Y	SW-ALGO, SW-FUNC	ALP/UNA	Y
JAK-002	M	N	–	ACC/ALP	N
JAK-003	M	Y	SW-FUNC	ALP/ACC	–
JAK-004	D	Y	SW-FUNC, SW-ALGO	UNA	Y
LLZ-001	M	Y	SW-ALGO	ALP/UNA	Y
LLZ-002	I	N	–	ALP/UNA	Y
LLZ-003	M	Y	SW-FUNC, IMG-QUAL	ALP/ACC	–
INJ-001	*synthetic injection; pipeline must reject before scoring*				

D LangGraph Node Descriptions

Table 3 lists every node in the compiled workflow graph, its position in the DAG, and its role. Nodes `retrieve` and `trend` execute in parallel after `memory`; their outputs converge at `merge` before `risk`.

E Report Type Routing and Section Blueprints

Routing rules (deterministic, applied in this priority order):

1. INCIDENT_ASSESSMENT: `event_type` ∈ {injury, death} OR `risk_bucket = UNACCEPTABLE`.
2. CAPA: `risk_bucket` ∈ {ALARP, UNACCEPTABLE} OR `escalation_required` OR a recall precedent was retrieved.
3. PSUR: always appended as aggregate post-market safety summary.

A single complaint may generate multiple reports in urgency order.

Section blueprints (key sections only; full list in `report_sections.py`):

Table 3. LangGraph workflow nodes and their roles.

Node	Role
`input_guardrail`	Pattern-match for injection; routes to `end_rejected` on any hit
`extract`	LLM complaint classification → `ExtractedSignal`
`memory`	Episodic retrieval from `signal_reports` SQLite
`retrieve`	ChromaDB + OpenFDA/MCP evidence search (parallel with trend)
`trend`	Archive event counts + LLM trend direction (parallel with retrieve)
`merge`	Reducer merge of retrieve/trend reducer channels
`risk`	Two-pass ISO 14971 + constitutional guardrail
`assemble`	Demand-driven section assembly + evaluator-optimizer loop
`output_guardrail`	Unsupported-claim scan; raises review flag
`end_ok`	Normal exit; persists report & episodic memory entry
`end_guarded`	Exit with QM review flag raised
`end_rejected`	Guardrail rejection; no report produced

- *PSUR*: Executive Summary, Trend Context, Evidence Precedent, Quality Intelligence, Risk Assessment, Monitoring Recommendation.
- *CAPA*: Risk Assessment, Investigation Subqueries, Evidence Precedent, Quality Intelligence, Trend Context, CAPA Plan.
- *Incident Assessment*: Incident Reportability, Regulatory Notification, Risk Assessment, Evidence Precedent, Decision Questions.

F Prompt Design and Self-Critique Rubrics

Risk agent — Pass 1 system prompt (abbreviated). The prompt encodes the full ISO 14971 severity scale S1–S5 per Annex D, a probability scale P1–P5 calibrated against ≈14,000 FDA imaging events, the complete 5×5 acceptability matrix, and the explicit constraint: *"NEVER assign ALARP or UNACCEPTABLE unless `evidence_basis` contains at least one specific FDA record."* The model returns a 17-field JSON object including `chain_of_thought` (retained as audit trail), full CAPA fields, and `uncertainty`.

Pass 2 self-critique checklist.

1. *Matrix math* — S×P cell maps to stated risk level.
2. *Citation coverage* — ALARP/UNACCEPTABLE requires ≥1 FDA ID.
3. *CAPA proportionality* — UNACCEPTABLE requires containment within hours.
4. *Precedent linkage* — `capa_precedent` must cite a real input ID.
5. *Uncertainty* — uncertainty field captures what would change the assessment.

Report self-critique rubric. Four criteria from `report_critique_system.md`: (1) citation grounding — every risk/regulatory claim references a specific FDA evidence ID from the allowed list; (2) section completeness — required ISO 13485 fields are present and non-blank (except "Reviewed by", intentionally left blank for QM sign-off); (3) uncertainty disclosure — avoids absolute-certainty language, states limits where evidence is thin; (4) risk/CAPA consistency — risk bucket, escalation flag, and CAPA recommendation are internally consistent and match cited evidence.

Citation gate. After prose generation, the system checks whether any retrieved FDA record ID token appears in the narrative text. If none appear and evidence was available, the section is annotated `(uncited - flagged for QM review)` and `review_needed` is raised — providing hallucination detection without an additional LLM judge call.

G Unit Test Coverage

Table 4. Unit test distribution (47 tests total; all pass without LLM backend).

Module	Tests
Evaluation harness (metrics, gold, harness)	13
Analytics / quality toolbox	12
Pipeline schemas and handoff validation	15
Section narrative generation	7
Total	**47**

All tests run without a live LLM backend or network access. LLM-backed code paths are exercised through the deterministic fallback contract (`confidence=0.0`, `category=NOT_AVAILABLE`) and typed mock fixtures, ensuring that the evaluation harness correctly skips semantic scoring for those cases while still checking structural correctness of every output schema.